

Experiment No:7 Write a simple python program to demonstrate various functions (Lambda).

In []: *#Function*

```
In [3]: def my_function():  
        print("Hello World")  
        my_function()
```

Hello World

```
In [4]: def my_function():  
        print("Hello World")  
        my_function()  
        my_function()  
        my_function()
```

Hello World

Hello World

Hello World

```
In [6]: temp1 = 77  
        celsius1 =(temp1 - 32) * 5 / 9  
        print(celsius1)  
  
        temp2 = 95  
        celsius2 =(temp2 - 32) * 5 / 9  
        print(celsius2)  
  
        temp3 = 50  
        celsius3 =(temp3 - 32) * 5 / 9  
        print(celsius3)
```

25.0

35.0

10.0

```
In [7]: def fahrenheit_to_celsius(fahrenheit):  
        return (fahrenheit - 32) * 5 / 9  
        print(fahrenheit_to_celsius(77))  
        print(fahrenheit_to_celsius(95))  
        print(fahrenheit_to_celsius(50))
```

25.0

35.0

10.0

In [8]: *#Return Values*

```
In [9]: def get_greeting():  
        return "Hello World"  
  
message = get_greeting()  
print(message)
```

Hello World

```
In [10]: def get_greeting():  
         return "Hello World"  
  
print(get_greeting())
```

Hello World

```
In [11]: def my_function():  
         pass
```

```
In [12]: #Arguments
```

```
In [14]: def my_function(fname):  
         print(fname + "Hello")
```

```
my_function("Bird")  
my_function("Animal")  
my_function("Nature")
```

BirdHello
AnimalHello
NatureHello

```
In [15]: #Parameters vs Arguments
```

```
In [16]: def my_function(name): # name is a parameter  
         print("Hello", name)  
  
my_function("Prachi") # "Emil" is an argument
```

Hello Prachi

```
In [17]: def my_function(fname, lname):  
         print(fname + " " + lname)  
  
my_function("Prachi", "Majalekar")
```

Prachi Majalekar

```
In [18]: def my_function(fname, lname):  
         print(fname + " " + lname)  
  
         my_function("Prachi")
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-18-629f69c7492b> in <module>  
      2     print(fname + " " + lname)  
      3  
----> 4 my_function("Prachi")  
  
TypeError: my_function() missing 1 required positional argument: 'lname'
```

```
In [22]: def my_function(name = "friend"):  
         print("Hello", name)  
  
         my_function("Prachi")  
         my_function("Riya")  
         my_function()  
         my_function("Rishu")
```

```
Hello Prachi  
Hello Riya  
Hello friend  
Hello Rishu
```

```
In [23]: def my_function(country = "India"):  
         print("I am from", country)  
  
         my_function("Brazil")  
         my_function("Russia")  
         my_function()  
         my_function("Sweden")
```

```
I am from Brazil  
I am from Russia  
I am from India  
I am from Sweden
```

```
In [ ]: #Keyword Arguments
```

```
In [28]: def my_function(animal, name):  
         print("I have a", animal)  
         print("My", animal + "'s name is", name)  
  
         my_function(animal = "Cat", name = "Money")
```

```
I have a Cat  
My Cat's name is Money
```

```
In [29]: def my_function(animal, name):  
         print("I have a", animal)  
         print("My", animal + "'s name is", name)  
  
         my_function(name = "Money", animal = "Cat")
```

I have a Cat
My Cat's name is Money

```
In [30]: def my_function(animal, name):  
         print("I have a", animal)  
         print("My", animal + "'s name is", name)  
  
         my_function("Cat", "Money")
```

I have a Cat
My Cat's name is Money

```
In [31]: def my_function(animal, name):  
         print("I have a", animal)  
         print("My", animal + "'s name is", name)  
  
         my_function("Money", "Cat")
```

I have a Money
My Money's name is Cat

```
In [32]: def my_function(animal, name, age):  
         print("I have a", age, "year old", animal, "named", name)  
  
         my_function("Cat", name = "Money", age = 4)
```

I have a 4 year old Cat named Money

```
In [33]: #Data Types
```

```
In [34]: def my_function(fruits):  
         for fruit in fruits:  
             print(fruit)  
  
         my_fruits = ["apple", "banana", "cherry"]  
         my_function(my_fruits)
```

apple
banana
cherry

```
In [36]: def my_function(person):  
         print("Name:", person["name"])  
         print("Age:", person["age"])  
  
         my_person = {"name": "Prachi", "age":19}  
         my_function(my_person)
```

Name: Prachi
Age: 19

```
In [37]: def my_function(x, y):  
         return x + y  
  
         result = my_function(3, 3)  
         print(result)
```

6

```
In [39]: def my_function():  
         return ["apple", "cherry", "banana"]  
  
         fruits = my_function()  
         print(fruits[0])  
         print(fruits[1])  
         print(fruits[2])
```

apple
cherry
banana

```
In [44]: def my_function():  
         return (20, 30)  
         x, y = my_function()  
         print("x:", x)  
         print("y:", y)
```

x: 20
y: 30

```
In [ ]: #Positional-Only Arguments
```

```
In [45]: def my_function(name, /):  
         print("Hello", name)  
  
         my_function("Prachi")
```

Hello Prachi

```
In [46]: def my_function(name):  
         print("Hello", name)  
  
         my_function("Prachi")
```

Hello Prachi

```
In [47]: def my_function(name, /):  
         print("Hello", name)  
  
         my_function(name = "Prachi")#this is the error part
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-47-2dd7cd63ba71> in <module>  
      2     print("Hello", name)  
      3  
----> 4 my_function(name = "Prachi")#this is the error part  
  
TypeError: my_function() got some positional-only arguments passed as keyword arguments: 'name'
```

```
In [48]: #Keyword-Only
```

```
In [49]: def my_function(*, name):  
         print("Hello", name)  
  
         my_function(name = "Prachi")
```

Hello Prachi

```
In [50]: def my_function(name):  
         print("Hello", name)  
  
         my_function(name = "Prachi")
```

Hello Prachi

```
In [51]: def my_function(*, name):
          print("Hello", name)

          my_function("Prachi")
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-51-cbde8a896801> in <module>
      2     print("Hello", name)
      3
----> 4 my_function("Prachi")

TypeError: my_function() takes 0 positional arguments but 1 was given
```

```
In [52]: def my_function(a, b, /, *, c, d):
          return a + b + c + d

          result = my_function(5, 10, c = 15, d = 20)
          print(result)
```

50

```
In [53]: # *args and **kwargs
```

```
In [1]: def my_function(*kids):
          print("The youngest child is " + kids[2])

          my_function("Prachi", "Riya", "Anu")
```

The youngest child is Anu

```
In [2]: def my_function(*args):
          print("Type:", type(args))
          print("First argument:", args[0])
          print("Second argument:", args[1])
          print("All arguments:", args)

          my_function("Prachi", "Riya", "Anu")
```

Type: <class 'tuple'>
 First argument: Prachi
 Second argument: Riya
 All arguments: ('Prachi', 'Riya', 'Anu')

```
In [3]: def my_function(greeting, *names):  
        for name in names:  
            print(greeting, name)  
  
my_function("Hello", "Prachi", "Riya", "Anu")  
  
Hello Prachi  
Hello Riya  
Hello Anu
```

```
In [4]: def my_function(*numbers):  
        total = 0  
        for num in numbers:  
            total += num  
        return total  
  
print(my_function(1, 2, 3))  
print(my_function(10, 20, 30, 40))  
print(my_function(5))  
  
6  
100  
5
```

```
In [6]: def my_function(*numbers):  
        if len(numbers) == 0:  
            return None  
        max_num = numbers[0]  
        for num in numbers:  
            if num > max_num:  
                max_num = num  
        return max_num  
  
print(my_function(3, 7, 2, 11, 1))  
  
11
```

```
In [7]: ***kwargs
```

```
In [9]: def my_function(**kids):  
        print("His last name is " + kids["lname"])  
  
my_function(fname = "Prachi", lname = "Riya")  
  
His last name is Riya
```

```
In [10]: def my_function(**myvar):
          print("Type:", type(myvar))
          print("Name:", myvar["name"])
          print("Age:", myvar["age"])
          print("All data:", myvar)

          my_function(name = "Riya", age = 19, city = "Kolhapur")
```

Type: <class 'dict'>
Name: Riya
Age: 19
All data: {'name': 'Riya', 'age': 19, 'city': 'Kolhapur'}

```
In [13]: def my_function(username, **details):
          print("Username:", username)
          print("Additional details:")
          for key, value in details.items():
              print(" ", key + ":", value)

          my_function("Riya123", age = 19, city = "Kolhapur", hobby = "Coding")
```

Username: Riya123
Additional details:
 age: 19
 city: Kolhapur
 hobby: Coding

```
In [14]: #Combine both
```

```
In [15]: def my_function(title, *args, **kwargs):
          print("Title:", title)
          print("Positional arguments:", args)
          print("Keyword arguments:", kwargs)

          my_function("User Info", "Prachi", "Riya", age = 20, city = "Kolhapur")
```

Title: User Info
Positional arguments: ('Prachi', 'Riya')
Keyword arguments: {'age': 20, 'city': 'Kolhapur'}

```
In [16]: def my_function(a, b, c):
          return a + b + c

          numbers = [1, 4, 3]
          result = my_function(*numbers)
          print(result)
```

```
In [18]: def my_function(fname, lname):  
         print("Hello", fname, lname)  
  
         person = {"fname": "Prachi", "lname": "Riya"}  
         my_function(**person)
```

Hello Prachi Riya

```
In [1]: #Scope
```

```
In [2]: #Local scope
```

```
In [3]: def myfunc():  
         x = 300  
         print(x)
```

```
myfunc()
```

300

```
In [5]: def myfunc():  
         x = 400  
         def myinnerfunc():  
             print(x)  
         myinnerfunc()
```

```
myfunc()
```

400

```
In [6]: #Global scope
```

```
In [7]: x = 400
```

```
def myfunc():  
    print(x)
```

```
myfunc()
```

```
print(x)
```

400

400

```
In [8]: x = 400

def myfunc():
    x = 300
    print(x)

myfunc()

print(x)
```

```
300
400
```

```
In [9]: def myfunc():
        global x
        x = 300

myfunc()

print(x)
```

```
300
```

```
In [10]: x = 400

def myfunc():
    global x
    x = 300

myfunc()

print(x)
```

```
300
```

```
In [11]: # NonLocal Keyword
```

```
In [12]: def myfunc1():
        x = "Jane"
        def myfunc2():
            nonlocal x
            x = "Hello World"
        myfunc2()
        return x

print(myfunc1())
```

```
Hello World
```

```
In [13]: x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print("Inner:", x)
    inner()
    print("Outer:", x)
outer()
print("Global:", x)
```

Inner: local
Outer: enclosing
Global: global

```
In [1]: #Decorators
```

```
In [2]: def changecase(func):
    def myinner():
        return func().upper()
    return myinner

@changecase
def myfunction():
    return "Hello World"

print(myfunction())
```

HELLO WORLD

```
In [3]: def changecase(func):
    def myinner():
        return func().upper()
    return myinner

@changecase
def myfunction():
    return "Hello World"

@changecase
def otherfunction():
    return "I am Prachi"

print(myfunction())
print(otherfunction())
```

HELLO WORLD
I AM PRACHI

```
In [4]: def changecase(func):
        def myinner(x):
            return func(x).upper()
        return myinner

@changecase
def myfunction(nam):
    return "Hello " + nam

print(myfunction("Riya"))
```

HELLO RIYA

```
In [5]: # *args and **kwargs
```

```
In [6]: def changecase(func):
        def myinner(*args, **kwargs):
            return func(*args, **kwargs).upper()
        return myinner

@changecase
def myfunction(nam):
    return "Hello " + nam

print(myfunction("Nidhi"))
```

HELLO NIDHI

```
In [7]: def changecase(n):
        def changecase(func):
            def myinner():
                if n == 1:
                    a = func().lower()
                else:
                    a = func().upper()
                return a
            return myinner
        return changecase

@changecase(1)
def myfunction():
    return "Hello World"

print(myfunction())
```

hello world

```
In [8]: def changecase(func):
        def myinner():
            return func().upper()
        return myinner

        def addgreeting(func):
            def myinner():
                return "Hello " + func() + " Have a good day!"
            return myinner

        @changecase
        @addgreeting
        def myfunction():
            return "Prachi"

        print(myfunction())
```

HELLO PRACHI HAVE A GOOD DAY!

```
In [9]: # metadata
```

```
In [14]: def myfunction():
          return "Have a great day!"

          print(myfunction.__name__)
```

myfunction

```
In [17]: def changecase(func):
          def myself():
              return func().upper()
          return myself

          @changecase
          def myfunction():
              return "Have a great day!"

          print(myfunction.__name__)
```

myself

```
In [18]: import functools

def changecase(func):
    @functools.wraps(func)
    def myinner():
        return func().upper()
    return myinner

@changecase
def myfunction():
    return "Have a great day!"

print(myfunction.__name__)
```

myfunction

```
In [1]: #python Lambda
```

```
In [2]: x = lambda a: a + 10
print(x(5))
```

15

```
In [3]: x = lambda a, b: a * b
print(x(5, 6))
```

30

```
In [4]: x = lambda a, b, c: a + b + c
print(x(5, 6, 2))
```

13

```
In [5]: def myfunc(n):
    return lambda a : a * n

mydoubler = myfunc(2)

print(mydoubler(11))
```

22

```
In [6]: def myfunc(n):  
        return lambda a : a * n  
  
        mytripler = myfunc(3)  
  
        print(mytripler(11))
```

33

```
In [7]: def myfunc(n):  
        return lambda a : a * n  
  
        mydoubler = myfunc(2)  
        mytripler = myfunc(3)  
  
        print(mydoubler(11))  
        print(mytripler(11))
```

22

33

```
In [8]: #Built in function
```

```
In [9]: numbers = [1, 2, 3, 4, 5]  
        doubled = list(map(lambda x: x * 2, numbers))  
        print(doubled)
```

[2, 4, 6, 8, 10]

```
In [10]: numbers = [1, 2, 3, 4, 5, 6, 7, 8]  
        odd_numbers = list(filter(lambda x: x % 2 != 0, numbers))  
        print(odd_numbers)
```

[1, 3, 5, 7]

```
In [11]: students = [("Emil", 25), ("Tobias", 22), ("Linus", 28)]  
        sorted_students = sorted(students, key=lambda x: x[1])  
        print(sorted_students)
```

[('Tobias', 22), ('Emil', 25), ('Linus', 28)]

```
In [12]: words = ["apple", "pie", "banana", "cherry"]  
        sorted_words = sorted(words, key=lambda x: len(x))  
        print(sorted_words)
```

['pie', 'apple', 'banana', 'cherry']

In [13]: *#Recursion*

```
In [14]: def countdown(n):  
    if n <= 0:  
        print("Done!")  
    else:  
        print(n)  
        countdown(n - 1)  
  
countdown(5)
```

5
4
3
2
1
Done!

```
In [15]: def factorial(n):  
    # Base case  
    if n == 0 or n == 1:  
        return 1  
    # Recursive case  
    else:  
        return n * factorial(n - 1)  
  
print(factorial(5))
```

120

```
In [16]: def fibonacci(n):  
    if n <= 1:  
        return n  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
print(fibonacci(7))
```

13

```
In [17]: def sum_list(numbers):  
    if len(numbers) == 0:  
        return 0  
    else:  
        return numbers[0] + sum_list(numbers[1:])  
  
my_list = [1, 2, 3, 4, 5]  
print(sum_list(my_list))
```

15

```
In [18]: def find_max(numbers):  
    if len(numbers) == 1:  
        return numbers[0]  
    else:  
        max_of_rest = find_max(numbers[1:])  
        return numbers[0] if numbers[0] > max_of_rest else max_of_rest  
  
my_list = [3, 7, 2, 9, 1]  
print(find_max(my_list))
```

9

```
In [19]: #Generator
```

```
In [20]: def my_generator():  
    yield 1  
    yield 2  
    yield 3  
  
for value in my_generator():  
    print(value)
```

1
2
3

```
In [21]: def count_up_to(n):  
    count = 1  
    while count <= n:  
        yield count  
        count += 1  
  
for num in count_up_to(5):  
    print(num)
```

1
2
3
4
5

```
In [22]: def large_sequence(n):
         for i in range(n):
             yield i

         # This doesn't create a million numbers in memory
         gen = large_sequence(1000000)
         print(next(gen))
         print(next(gen))
         print(next(gen))
```

0
1
2

```
In [23]: def simple_gen():
         yield "Emil"
         yield "Tobias"
         yield "Linus"

         gen = simple_gen()
         print(next(gen))
         print(next(gen))
         print(next(gen))
```

Emil
Tobias
Linus

```
In [24]: def simple_gen():
         yield 1
         yield 2

         gen = simple_gen()
         print(next(gen))
         print(next(gen))
         print(next(gen)) # This will raise StopIteration
```

1
2

```
-----
StopIteration                                Traceback (most recent call last)
Cell In[24], line 8
      6 print(next(gen))
      7 print(next(gen))
----> 8 print(next(gen)) # This will raise StopIteration

StopIteration:
```

```
In [25]: #Expressions
```

```
In [26]: # List comprehension - creates a list
list_comp = [x * x for x in range(5)]
print(list_comp)

# Generator expression - creates a generator
gen_exp = (x * x for x in range(5))
print(gen_exp)
print(list(gen_exp))
```

```
[0, 1, 4, 9, 16]
<generator object <genexpr> at 0x000001C8EBB159E0>
[0, 1, 4, 9, 16]
```

```
In [27]: # Calculate sum of squares without creating a list
total = sum(x * x for x in range(10))
print(total)
```

```
285
```

```
In [28]: def fibonacci():  
         a, b = 0, 1  
         while True:  
             yield a  
             a, b = b, a + b  
  
         # Get first 100 Fibonacci numbers  
         gen = fibonacci()  
         for _ in range(100):  
             print(next(gen))
```

0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
1597
2584
4181
6765
10946
17711
28657
46368
75025
121393
196418
317811
514229
832040
1346269
2178309
3524578
5702887
9227465
14930352
24157817
39088169
63245986
102334155
165580141
267914296
433494437
701408733
1134903170
1836311903
2971215073
4807526976
7778742049
12586269025
20365011074
32951280099
53316291173
86267571272
139583862445
225851433717

365435296162
591286729879
956722026041
1548008755920
2504730781961
4052739537881
6557470319842
10610209857723
17167680177565
27777890035288
44945570212853
72723460248141
117669030460994
190392490709135
308061521170129
498454011879264
806515533049393
1304969544928657
2111485077978050
3416454622906707
5527939700884757
8944394323791464
14472334024676221
23416728348467685
37889062373143906
61305790721611591
99194853094755497
160500643816367088
259695496911122585
420196140727489673
679891637638612258
1100087778366101931
1779979416004714189
2880067194370816120
4660046610375530309
7540113804746346429
12200160415121876738
19740274219868223167
31940434634990099905
51680708854858323072
83621143489848422977
135301852344706746049
218922995834555169026

```
In [29]: def echo_generator():  
    while True:  
        received = yield  
        print("Received:", received)  
  
    gen = echo_generator()  
    next(gen) # Prime the generator  
    gen.send("Hello")  
    gen.send("World")
```

Received: Hello
Received: World

```
In [31]: def my_gen():  
    try:  
        yield 1  
        yield 2  
        yield 3  
    finally:  
        print("Generator closed")  
  
    gen = my_gen()  
    print(next(gen))  
    gen.close()
```

1
Generator closed

```
In [32]: # Exercise
```

```
In [33]: ##### def check_even_odd(n):  
    if n % 2 == 0:  
        print("Even number")  
    else:  
        print("Odd number")  
  
    n = int(input("Enter a number: "))  
  
    check_even_odd(n)
```

Enter a number: 44
Even number

```
In [34]: def sum_three(a, b, c):  
        total = a + b + c  
  
        if a == b == c:  
            return total * 3  
        else:  
            return total  
  
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
c = int(input("Enter third number: "))  
  
print("Result:", sum_three(a, b, c))
```

Enter first number: 3
Enter second number: 3
Enter third number: 3

Result: 27

```
In [35]: #Lambda
```

```
In [36]: a = int(input("Enter first number: "))  
        b = int(input("Enter second number: "))  
  
        sum = lambda x, y: x + y  
  
        print("Sum:", sum(a, b))
```

Enter first number: 2
Enter second number: 5

Sum: 7

```
In [ ]:
```